NNUI_CELL

A Continuum Field Model for Self-Settling User Interfaces
Governed by a Rheological Constitutive Law and a Genotype–Phenotype Expression Layer

Proof of Concept & Formal Specification — v0.1

R. Forrestall ("Heat")
MIVIM Development — *Standards in Novel Design*

July 2026

Abstract

Contemporary UI is authored as a large set of independent boxes, each carrying private geometry and style. Positions are hand-placed or solved by ad-hoc constraint systems, and adaptation to new content, new screens, or agent-driven edits requires re-authoring geometry. We propose **NNUI_CELL**, a model that replaces the box abstraction with a single *continuous medium* whose layout is never authored but always *solved* from local material state. The medium is governed by two interlocking formal axes. The **rheological axis** treats every region as yield-stress fluid governed by a solidity-modulated Herschel–Bulkley constitutive law with a rate-dependent yield and thixotropic settling; layout equilibrium ("settled") is defined as an energy minimum reached by overdamped gradient flow and terminated by a strain-rate threshold. The **genetic axis** treats every region as a cell carrying a *genome* (absorbed data traits) that deterministically *expresses* a *phenotype* (rendered controls and skin) through an extensible registry. The two axes share a single interface primitive: a boundary carries an always-on material-diffusion coefficient and an opt-in, deterministic gene-transfer operation. We give the constitutive derivations, the discrete real-time solver (XPBD with viscosity-modulated compliance), a construction for bit-identical determinism, the genetic operator set (injection, mitosis, symbiotic binding), a compact declarative grammar intended for LLM/agent emission, and seven executable invariants that pin the design against reductive drift. We close with four designed proof-of-concept experiments and an honest account of limitations. This document specifies a system; it does not report a completed implementation.

---

1 Motivation: the box model is the wrong primitive

The dominant UI abstraction is a tree of independent rectangles. Each node owns its box, its style, and — through cascade, flexbox, or grid — a share of the negotiation over space. Three costs follow directly from the primitive itself, not from any particular framework:

1. **Assets clash.** Two components were never the same substance, so when they meet they overlap, collide, or must be manually reconciled. Integration is glue work.
2. **Layout precedes design.** The pipeline is scaffold → fit → lay out → *then* design. Structure and appearance are separate passes because boxes hold both independently.
3. **Adaptation re-authors geometry.** New content, a resized viewport, or an agent edit re-invokes an authoring step. For an autonomous agent this means emitting and iterating explicit coordinates — expensive in tokens and brittle under change.

NNUI_CELL removes the primitive. There are no independent boxes. There is one continuous medium, and a "component" is a region of that medium distinguished only by its local parameter field and its skin. Every consequence in this paper is a consequence of that single move; the rheology and the biology are the two mechanisms that make a continuum tractable and addressable.

1.1 Contributions

- A **solidity-modulated Herschel–Bulkley** constitutive law (§4.1) that folds "yield stress," "consistency," and "locked" onto a single continuous field $S \in [0, 1]$, so that locking is the top of the resistance dial rather than a separate mechanism.
- A **rate-dependent yield** formulation (§4.2) that resolves an apparent contradiction in the design intent — a material cannot be both shear-thinning and shear-thickening — by placing the rate dependence in the *yield*, not the flow index.
- A definition of **settling** as overdamped gradient flow of an explicit energy functional (§4.5), terminated by a measurable strain-rate threshold.
- **Viscosity-modulated XPBD compliance** (§5): the bridge that turns the continuum law into a stable 60 fps solver by setting each constraint's compliance from the local apparent viscosity. This is the engineering core.
- A deterministic **genotype→phenotype** expression layer (§6) with an extensible registry, and a reduction of the biological operations (injection, mitosis, binding) to defined, non-stochastic mechanisms.
- A **unified boundary operator** (§6.4) carrying two decoupled coefficients, so physical proximity never forces genetic fusion.
- Seven **executable invariants** (§8) that convert anti-drift intent into automated acceptance tests.

2 Positioning and prior art

We separate what exists from what we claim is new, to keep the novelty statement honest.

- **Spring / physics-flavoured animation** (iOS UIKit dynamics, Framer Motion, React Spring) animates *between* authored layouts using easing or spring presets. Layout itself is still box-authored, and the physics is decorative. NNUI_CELL has no animation system at all (Invariant 6); motion is the solver relaxing, never a tween.
- **Constraint layout** (Cassowary/AutoLayout, CSS flex/grid) solves geometry from declared relations. This is the closest neighbour. The difference is the governing law: constraint solvers optimize algebraic relations to a unique satisfying assignment; NNUI_CELL evolves a *material* under a rheological law toward an energy minimum, giving it settling dynamics, yield behaviour, locking, and content-preserving flow as one continuum rather than as separate features.
- **Soft-body / position-based dynamics** (PBD/XPBD in games) simulates deformable matter. We reuse the solver but drive its parameters from a rheological constitutive law and attach a semantic (genetic) layer, which PBD does not have.
- **Blob / metaball visual effects** are rendering tricks with no layout semantics.

- **Data-driven component selection** (schema-to-widget mappers, BI auto-charting) resembles our expression layer but is not embedded in a shared physical medium and has no notion of trait inheritance across a shared interface.

Novelty claim (defensible): *a layout engine whose constitutive law is an actual rheological equation, in which authors and agents parameterize material properties and containment rather than coordinates, unified with a deterministic genotype→phenotype expression layer at a single interface primitive.* We are unaware of prior art combining these.

3 The two-axis model

A region of the medium — a *cell* — is fully specified only by two orthogonal axes.

Axis 1 — Rheological *how it flows*	**Axis 2 — Genetic** *what it becomes*
Geometry over time: position, motion, settling, locking, boundaries. State: S, n, ξ (per point); D, bind (per interface). Mechanism: solidity-modulated Herschel–Bulkley; energy-minimizing settling.	Identity and manifestation: what data a cell holds and which controls it grows. State: genome Γ; phenotype $\Phi = \text{express}(\Gamma)$. Mechanism: sequencing seq; deterministic expression via registry $\mathcal{R}$.

Design principle. The rheological axis alone cannot say how a dropped CSV becomes a chart; the genetic axis alone cannot say where the chart goes or how it settles. The model's expressive power is the product, not the sum, of the two axes.

4 Rheological axis: constitutive model

4.1 Solidity-modulated Herschel–Bulkley law

The classical Herschel–Bulkley (HB) model describes a material that is rigid below a yield stress τ_y and flows above it:

$$\tau = \begin{cases} \text{rigid}, & \tau \le \tau_y, \\ \tau_y + K\,\dot\gamma^{\,n}, & \tau > \tau_y, \end{cases} \tag{1}$$

with consistency K and flow index n ($n < 1$ shear-thinning, $n > 1$ shear-thickening), $\dot\gamma$ the shear rate.

We introduce a single *solidity* field $S(\mathbf{x}) \in [0, 1)$ that jointly modulates yield and consistency, so a single dial spans "freely flowing" to "locked":

$$\tau_y(S) = \tau_y^0 \left(\frac{S}{1-S}\right)^{\kappa}, \tag{2}$$

$$K(S) = K^0\,(1 + \beta S), \tag{3}$$

with base scale τ_y^0, shaping exponent $\kappa > 0$, and consistency gain $\beta \ge 0$. The pole at $S = 1$ in (2) is deliberate: $\tau_y \to \infty$ as $S \to 1$, so no finite input can yield the region — **locking is the limit** $S \to 1$, not a separate flag. This collapses the three lock tools into one primitive, `set_solidity(mask,S)` (§8.1).

For numerical use we adopt the Papanastasiou regularization of the yield term, which replaces the discontinuity by a steep smooth transition governed by a regularization parameter m:

$$\eta_{\text{app}}(\dot\gamma, S) \;=\; \tau_y(S)\,\frac{1 - e^{-m\dot\gamma}}{\dot\gamma} \;+\; K(S)\,\dot\gamma^{\,n-1}, \tag{4}$$

where η_{app} is the apparent viscosity relating stress to rate, $\tau = \eta_{\text{app}}\dot{\gamma}$. Equation (4) is the quantity the solver consumes.

4.2 The dual-rate response: rate-dependent yield

The design intent contains an apparent contradiction. It asks for (a) rigidity by default; (b) a fast, accidental impulse to *stiffen* and snap (Oobleck-like, shear-thickening); and (c) a slow, deliberate press to *soften* and flow (reverse-Oobleck, shear-thinning). A single flow index n cannot be both < 1 and > 1.

The resolution is to separate the two effects onto different terms. Post-yield flow is shear-thinning ($n < 1$: once you have committed, the material gets out of your way). The impulse-stiffening lives not in n but in a *rate-dependent yield*:

$$\tau_y^{\text{eff}}(S, \xi, \dot{\gamma}) \;=\; \underbrace{\tau_y^0 \Big(\tfrac{S}{1-S}\Big)^{\kappa}}_{\text{solidity}} \cdot \underbrace{\xi}_{\text{structure}} \cdot \underbrace{\big(1 + (\dot{\gamma}/\dot{\gamma}_c)^q\big)}_{\text{impact hardening}}, \tag{5}$$

with critical rate $\dot{\gamma}_c$ and hardening exponent $q > 0$; ξ is the thixotropic structure parameter of §4.3. A slow deliberate input ($\dot{\gamma} \ll \dot{\gamma}_c$) sees $\tau_y^{\text{eff}} \approx \tau_y(S)\xi$ and, if sustained, exceeds it and flows smoothly (thinning). A fast flick ($\dot{\gamma} \gg \dot{\gamma}_c$) sees the yield spike by $(\dot{\gamma}/\dot{\gamma}_c)^q$, so the impulse is rejected and the region reads as solid. This is a rate-dependent yield-stress fluid — real rheology — and it is the honest unification the flat model hid.

4.3 Thixotropic settling: the "ooze"

Settling requires viscosity to *rebuild* once input stops. We attach a structure parameter $\xi(\mathbf{x}, t) \in [0, 1]$ with standard breakdown/rebuild kinetics:

$$\frac{d\xi}{dt} \;=\; \frac{1}{T_r}\,(1 - \xi) \;-\; a\,\xi\,\dot{\gamma}, \tag{6}$$

rebuild timescale T_r, breakdown coefficient a. Under shear the second term dominates and structure erodes ($\xi \downarrow$, material softens and flows); at rest the first term restores $\xi \to 1$ over $\sim T_r$, raising the effective yield through (5) until motion falls below threshold. The visible "moves around, then settles into the bowl" is exactly this relaxation. T_r is the "ooze" dial; the design notes' thixotropy dial λ is this ξ kinetics.

4.4 Content as rigid inclusions

Text and images must never shear when their region flows. We model each content payload as a *rigid inclusion* suspended in the yield-stress matrix: only the matrix deforms; the inclusion translates and rotates as a rigid body. Formally, inclusion i contributes a deformation penalty (§4.5) enforced as a rigid shape-matching constraint in the solver (§5), guaranteeing a paragraph moves without smearing.

4.5 Settling as overdamped gradient flow

Let $\mathbf{q}$ be the medium's configuration (discretized in §5). The rest state is the minimizer of a layout energy

$$E(\mathbf{q}) \;=\; \underbrace{w_f \int_\Omega \big(1 - \rho(\mathbf{x})\big)^2 d\mathbf{x}}_{E_{\text{fill}}} \;+\; \underbrace{w_c \sum_{(i,j)\in\partial} \sigma_{ij}\,\ell_{ij}}_{E_{\text{cohesion}}} \;+\; \underbrace{w_r \sum_i \big\|\mathbf{F}_i - \mathbf{R}_i\big\|_F^2}_{E_{\text{content}}}, \tag{7}$$

where ρ is occupancy over the container Ω (so E_{fill} penalizes voids — the slime fills the bowl); σ_{ij} is interfacial tension along boundary segment of length ℓ_{ij} (so E_{cohesion} minimizes interface length, producing coalescence and menisci); and for each inclusion $\mathbf{F}_i$ is the deformation gradient with $\mathbf{R}_i$ its best-fit rotation (so E_{content} penalizes any non-rigid deformation of content). The weights w_f, w_c, w_r are global design parameters.

The medium follows overdamped dynamics — inertia is dominated by the rheological dissipation, so we integrate a damped descent

$$\mathbf{M}\,\dot{\mathbf{q}} \;=\; -\nabla E(\mathbf{q}) \;-\; \mathbf{C}(\eta_{\text{app}})\,\dot{\mathbf{q}}, \tag{8}$$

with mass matrix $\mathbf{M}$ and viscous damping $\mathbf{C}$ set by the local apparent viscosity. **Settled** is a measurable predicate, not a duration:

$$\text{settled} \iff \|\dot{\gamma}\|_\infty < \varepsilon_{\text{settle}}. \tag{9}$$

Relaxation is asymptotic and never clamped; rendering is cut below a perceptual threshold. The "alive" quality is precisely this un-clamped decay — never a snap.

4.6 Assimilation as controlled diffusion

When a new region is injected, its material-parameter field $\phi = (S, n, \xi, \dots)$ blends into its neighbours by diffusion across the shared boundary:

$$\frac{\partial \phi}{\partial t} \;=\; \nabla\cdot\big(D(\mathbf{x})\,\nabla\phi\big), \tag{10}$$

with assimilation coefficient field D. Large D: the region melts in within a few frames. $D = 0$: it never blends — a permanent distinct island (modals, pinned widgets). Note (10) diffuses *material parameters*, not content; content is a rigid inclusion (§4.4) and does not diffuse. The discrete explicit scheme $\phi^{t+1} = \phi^t + \Delta t\, D\, \nabla_h^2 \phi^t$ is stable under the diffusion CFL condition $\Delta t \le h^2/(2d\,D)$ in d dimensions with grid spacing h.

5 Discretization and the real-time solver

5.1 Why XPBD

Full Navier–Stokes or SPH would be accurate and unaffordable, and its results are not needed: we want stable, controllable, real-time relaxation, not physical fidelity. Extended Position-Based Dynamics (XPBD) provides unconditional stability, real-time performance on modest hardware, and — critically — a per-constraint *compliance* that we can drive from the constitutive law. The medium is discretized as clusters of particles connected by constraints (distance, volume/occupancy, rigid shape-matching for inclusions, interface tension).

5.2 Viscosity-modulated compliance (the bridge)

XPBD solves each constraint C_j with the update

$$\Delta\lambda_j \;=\; \frac{-\,C_j(\mathbf{x}) - \tilde{\alpha}_j\,\lambda_j}{\nabla C_j\,\mathbf{M}^{-1}\,\nabla C_j^\top + \tilde{\alpha}_j}, \qquad \Delta\mathbf{x} \;=\; \mathbf{M}^{-1}\nabla C_j^\top\,\Delta\lambda_j, \tag{11}$$

where λ_j is the constraint's Lagrange multiplier and $\tilde{\alpha}_j = \alpha_j/\Delta t^2$ is its compliance. The central move of this paper is to set compliance from the local apparent viscosity (4):

$$\alpha_j \;=\; \frac{1}{k_0\, g\big(\eta_{\text{app}}(S,\xi,\dot{\gamma})\big)}, \qquad g \text{ monotone increasing.} \tag{12}$$

As $S \to 1$, $\eta_{\text{app}} \to \infty$, so $\alpha_j \to 0$ and the constraint becomes perfectly rigid — the region is *locked*. As $S \to 0$ the constraint is highly compliant — the region *flows*. Thus one field S, through (4)–(12), spans the entire range from liquid to locked, per constraint, per frame. Rigid content inclusions use $\alpha_j \to 0$ shape-matching constraints, realizing E_{content}.

5.3 The solver loop

Algorithm 1: One deterministic solver tick

Input: particles $\mathbf{x}$, velocities $\mathbf{v}$, fields (S, n, ξ, D), constraints $\{C_j\}$
Output: next-frame $\mathbf{x}', \mathbf{v}'$

$\mathbf{x}^p \leftarrow \mathbf{x} + \Delta t\, \mathbf{v}$ `// predict (no external gravity; layout forces only)`
estimate local shear rate $\dot{\gamma}$ from the predicted strain
update structure ξ by (6); update η_{app} by (4)
diffuse fields ϕ by (10) (assimilation)
for *iter* $= 1 \dots N_{solve}$ **do**
  foreach *constraint* C_j ***in fixed sorted order*** **do**
    compute compliance $\tilde{\alpha}_j$ by (12)
    apply XPBD update (11) to $\mathbf{x}^p$
  end
end
$\mathbf{v}' \leftarrow (\mathbf{x}^p - \mathbf{x})/\Delta t$; $\mathbf{x}' \leftarrow \mathbf{x}^p$
if $\|\dot{\gamma}\|_\infty < \varepsilon_{settle}$ **then**
  mark region **settled**; freeze rendering
end

5.4 Determinism by construction

Bit-identical settling (experiment R2) is required for testing, accessibility, and agent trust. It is achieved, not hoped for: (i) fixed timestep Δt; (ii) constraints solved in a fixed order sorted by stable identifier (Gauss–Seidel, no data races); (iii) all reductions performed in a fixed, sorted order so floating-point addition is associative-stable; (iv) no wall-clock or uninitialized state enters the update; (v) where cross-hardware reproducibility is needed, fixed-point or a fixed float evaluation order is mandated. Under these, identical inputs yield identical settled states on every run and machine.

6 Genetic axis: identity and expression

6.1 Genome, germline, sequencing

A cell carries a *genome* Γ, a set of typed *traits* (capability tokens such as `tabular`, `temporal`, `media:video`, `numeric-range`). Every cell is born with a *germline* Γ_0 expressing primitive phenotypes (bare panel, container, control surface); **there is no empty cell**, only a cell whose genome is Γ_0. This removes any special case for "empty" and makes the shipped default UI the same substance as user-created UI. Absorbing an asset *adds* genes:

$$\text{seq} : \text{Bytes} \to 2^{\text{Trait}}, \qquad \Gamma \;\leftarrow\; \Gamma \cup \text{seq}(\text{asset}). \tag{13}$$

seq is a pure function: MIME/structure/column-type detection in, trait-set out. No physics, no ambiguity, fully testable.

6.2 Expression as a deterministic registry map

Phenotype is a deterministic function of genome through an extensible registry $\mathcal{R} : \text{TraitSignature} \rightharpoonup \text{PhenotypeConstructor}$:

$$\Phi \;=\; \text{express}(\Gamma) \;=\; \text{compose}\left\{\, \mathcal{R}(\text{sig}) \;:\; \text{sig} \in \text{signatures}(\Gamma) \,\right\}. \tag{14}$$

A tabular genome expresses a chart phenotype; a video genome expresses a player phenotype. Extensibility is the whole plug-and-play claim made concrete: a new data type is a new entry in $\mathcal{R}$, added *without touching the core* (experiment R4). Expression is deterministic — there is no stochastic or mutation operator anywhere in the model.

6.3 Operations reduced to mechanism

Per Invariant 7, each biological term is admitted only after reduction to a defined mechanism.

Algorithm 2: Genetic operators

Injection inject(cell, asset):
$\Gamma \leftarrow \Gamma \cup \text{seq}(\text{asset})$; $\Phi \leftarrow \text{express}(\Gamma)$
set assimilation field D on the new boundary; rheology renders the blend via (10)

Mitosis mitose(cell):
$\Gamma_{\text{child}} \leftarrow \text{DEEP-COPY}(\Gamma_{\text{parent}})$ `// identity copied`
child inherits **no** position or settled geometry
$\Phi_{\text{child}} \leftarrow \text{express}(\Gamma_{\text{child}})$; child re-settles under its own local pressure
division visual = one mass pinching in two under interface tension (free from §4.5)

Binding bind(A, B) `// deliberate, deterministic`
on the marked shared interface: $\Gamma_A \leftarrow \Gamma_A \cup \tau(\Gamma_B)$, τ a deterministic (possibly partial) transfer
adjacency alone **never** triggers this

Mitosis is not "make a copy." Taken literally it is genome replication plus independent re-expression: a duplicated chart dropped into a cramped corner settles compact while its parent stays wide — same genotype, different phenotype-in-context. The naive "duplicate" verb would have hidden this correct behaviour.

6.4 The unified boundary operator

A boundary between two cells carries two *decoupled* coefficients:

Coefficient	Governs	Default
D (material diffusion)	do the regions blend physical material	always > 0 (one medium)
bind (gene transfer)	do the regions exchange genome	**off**; opt-in only

This preserves the elegance of "one interface, two jobs" while ensuring physical proximity during settling never forces genetic fusion. Symbiotic binding is precisely the deliberate act of engaging the second coefficient. There is no ambient, proximity-driven, or random gene exchange: such behaviour would violate determinism (R2) and Invariant 7 and is explicitly excluded.

7 The agent interface: declarative grammar and token economy

The end goal is an interface an LLM/agent emits cheaply. An agent declares *material intent and containment*, not coordinates; the solver produces and maintains geometry. The grammar has two verb families — rheological and genetic — over one region model.

```
program := { region | op } ;
region := "cell" id "{" material content "}" ;
material := "S:" num "n:" num "xi:" num "D:" num [ "skin:" id ] ;
content := "hold" asset-ref | "germline" ;
op := "inject" id asset-ref
           | "solidity" mask num // t1/t2/t3 all reduce here
           | "mitose" id
           | "bind" id id
           | "contain" id "in" region-ref ;
mask := "all" | id | "not" id ; // whole / selection / complement
```

Worked example (dashboard). An agent asks for a wide revenue panel that fills remaining space with three cohesive KPI cells grouped beside it:

```
cell main { S:0.3 n:0.8 xi:1 D:0.6 hold revenue.csv } // expresses -> chart
cell k1 { S:0.5 n:0.8 xi:1 D:0.4 hold kpi1 }
cell k2 { S:0.5 n:0.8 xi:1 D:0.4 hold kpi2 }
cell k3 { S:0.5 n:0.8 xi:1 D:0.4 hold kpi3 }
bind k1 k2 ; bind k2 k3 // cohesive group
contain { main k1 k2 k3 } in viewport
```

No coordinates appear. Positions, sizes, the chart type, and the group packing are *solved*. When `revenue.csv` changes or the viewport resizes, the medium re-settles with *no further agent call* — the zero-cost-adaptation claim. The economic argument is therefore not merely fewer tokens at generation ($\mathcal{O}(\text{dials} + \text{containment})$ versus $\mathcal{O}(\text{explicit geometry})$) but the elimination of the re-authoring round-trip on every change.

Honesty flag. The token advantage above is argued from structure, not yet measured. R3 (§9) exists to measure it against equivalent hand-authored markup on three real layouts; the claim is a hypothesis under test, not a result.

8 Invariants as executable specification

Anti-drift is enforced by artifacts, not intentions. Each invariant is stated as a predicate that becomes an automated acceptance test; a build that fails any is, by definition, drift.

I1. No authored positions. No code path assigns a coordinate directly; every position is a solver output. *Test:* static check + perturbation test (positions change iff material state changes).

I2. Real constitutive law. η_{app} is evaluated per frame from (4); changing n at runtime changes observed behaviour. *Test:* sweep n, assert response monotonicity; assert no easing/spring preset in the motion path.

I3. Locks are solidity. No boolean lock flag; all locking flows through `set_solidity`. *Test:* grep/AST for lock special-casing returns empty; $S = 1$ region is unyielding under bounded input.

I4. Threshold-terminated settling. Settling ends by (9), never by a timer. *Test:* varying content mass changes settle time; no duration constant gates it.

I5. One medium. No component owns private layout logic outside the solver. *Test:* removing a component type does not remove any layout code path.

I6. No animation system. The renderer is a stateless view of solver state; no tweens, timelines, or durations exist. *Test:* freezing the solver freezes all on-screen motion exactly.

I7. Metaphor reduces to mechanism. Every biological term in the codebase maps to a defined operator (§6); expression and all operators are deterministic. *Test:* R2 bit-identity holds; no stochastic call in the genetic path.

8.1 Tool primitives

The three authoring tools are not three features; they are three masks over one primitive:

Tool	Meaning	Reduction
`slime-lock` (t1)	lock whole UI solid	`set_solidity(all, 1)`
`window-lock` (t2)	lock a selected area (inside frozen)	`set_solidity(sel, 1)`
`isolation` (t3)	work inside a region (outside frozen)	`set_solidity(not sel, 1)`

Partial locks ($S = 0.8$: stiff but yielding to determined force) and soft gradient lock edges fall out for free, since S is a continuous field.

9 Proof-of-concept plan: designed experiments

Theory is frozen at this document. The remaining questions are empirical and constitute the proof of concept. Each has a binary exit criterion.

ID	Question	Exit criterion
R1	Discretization: does XPBD (§5) settle at real-time scale?	Headless Rust prototype settles 50 regions in < 16 ms/frame with the constitutive law of §4.1.
R2	Determinism (§5.4).	Bit-identical settled state across 100 runs (and across two machines).
R3	Agent grammar / token economy (§7).	Three real layouts (dashboard, article, settings) each expressed under ∼80 tokens and shown to settle unambiguously; token count compared to equivalent markup.
R4	Expression registry (§6).	Three data types (tabular, video, plain text) sequenced and expressed to distinct phenotypes headlessly; a fourth added without core modification.

R1 and R2 share one prototype. R3 and R4 are paper-and-pencil / headless software and run in parallel. **Deferred by design** (stage-5 questions, excluded to prevent scope drift): OS integration, multi-user editing, CSS interop, and performance beyond the 50-region target. Recommended build order per the project pipeline: this specification → R1+R2 prototype crate (headless, no UI) → R3/R4 in parallel → visual demo (workstream B) proving the ooze → agent-facing API (workstream A, the goal).

10 Limitations and threats to validity

We state these plainly; a PoC that hides them is not professional.

- **No implementation yet.** This specifies a system. Every performance and determinism claim is a hypothesis with an exit criterion, not a measured result.
- **Token economy is projected.** The agent-cost advantage is argued structurally (§7); R3 must measure it. It may be smaller than argued on layouts where explicit control is genuinely needed.

- **Cross-hardware float determinism is hard.** (11) over IEEE floats can diverge across architectures; R2's two-machine clause may force fixed-point, with a performance cost not yet quantified.
- **Rheological realism vs. performance is untuned.** The regularization m, solve iterations N_{solve}, and CFL bound trade stability against speed; the operating point is unknown until R1.
- **Accessibility is an open design, but a favourable one.** A continuous medium has no DOM semantics for assistive tech — however, the *genome* is a clean semantic layer (traits and expressed phenotype), arguably a better accessibility source than inferred box structure. This must be designed, not assumed.
- **Continuum cost at scale.** Memory and solve cost for very large interfaces ($\gg 50$ regions) is unaddressed and deliberately deferred.

11 Conclusion

NNUI_CELL proposes a single structural break — *replace the box model with a continuum field* — and supplies the two mechanisms that make a continuum both movable and addressable: a solidity-modulated, rate-dependent-yield Herschel–Bulkley rheology that settles by energy minimization and locks at the top of one dial, and a deterministic genotype→phenotype expression layer that lets a region digest and manifest arbitrary data. The two axes meet at one interface primitive with decoupled physical and genetic coupling, so proximity never forces fusion and every biological term reduces to a defined, non-stochastic mechanism. The design is pinned against reductive drift by seven executable invariants, and its open questions are stated as four designed experiments with binary exit criteria. What remains is not more theory but the prototype that will accept or refute it.

MIVIM Development — Standards in Novel Design. Specification v0.1, July 2026. Technical architecture developed in an AI-assisted research collaboration; all design authority and creative direction held by the author. This document is a proof-of-concept specification and does not report a completed implementation.

A Symbol and dial reference

Symbol	Meaning	Role
$S \in [0, 1)$	solidity field	dial: liquid→locked; per point
n	flow index	dial: post-yield thinning (< 1) / thickening (> 1)
$\xi \in [0, 1]$	thixotropic structure	state: erodes under shear, rebuilds at rest
T_r	rebuild timescale	dial: "ooze" / settle speed
D	assimilation coefficient	dial: melt-in (> 0) vs. island (0); per interface
$q, \dot{\gamma}_c$	impact-hardening exponent, critical rate	tune fast-flick rigidity
κ, β	yield shaping, consistency gain	calibrate the S curve
w_f, w_c, w_r	energy weights	fill vs. cohesion vs. content rigidity
$\varepsilon_{\text{settle}}$	strain-rate threshold	defines "settled"
$\alpha_j, \tilde{\alpha}_j$	XPBD compliance	set from η_{app} per constraint
Γ, Γ_0, Φ	genome, germline, phenotype	genetic state
$\mathcal{R}$	expression registry	extensible trait→phenotype map

The full per-point material state is $\{S, n, \xi, \text{skin}\}$; the per-interface state is $\{D, \text{bind}\}$. This compactness is the token-economy argument: a handful of scalars and a containment relation describe what box markup approximates in hundreds of declarations.